

SEntFiN Financial Sentiment Classifier

Entity-Aware Sentiment Analysis using FinBERT

Scholar No: 24u021017

Project Report — Data Mining & Warehousing

Abstract

This project fine-tunes the FinBERT transformer for fine-grained, entity-level sentiment classification of financial news headlines. Each input pairs a headline with a target financial entity, and the model predicts one of three sentiment labels (positive, negative, neutral) specific to that entity. The model is trained on the SEntFiN 1.0 dataset (Sinha et al., 2022), which provides 14405 entity-level annotations across 10,753 headlines. We report accuracy and macro-F1 on a held-out test set, along with per-class precision/recall.

1. Introduction

Sentiment analysis of financial news is critical for algorithmic trading, risk monitoring, and portfolio analytics. Headline-level sentiment is too coarse: a single headline often mentions multiple entities with conflicting sentiment (e.g., 'Gold rallies as Silver dips' is positive for Gold but negative for Silver). SEntFiN 1.0 provides entity-level annotations that enable a fine-grained model capable of disentangling these cases.

1.1 Problem Formulation

Given a headline H and an entity E mentioned within it, predict the sentiment polarity $y \in \{\text{negative, neutral, positive}\}$ that the headline expresses toward E .

$$y^* = \underset{c \in C}{\operatorname{argmax}} P(c \mid H, E; \theta) \quad (1)$$

2. Dataset: SEntFiN 1.0

SEntFiN 1.0 contains 10,753 financial news headlines collected from Indian and international financial news sources. Each headline is annotated with one or more (entity, sentiment) pairs stored as a Python-dict string in the 'Decisions' column.

Field	Value
Source	Sinha et al., 2022 (arXiv:2305.12257)
Headlines	10,753
Expanded triples	14405
Classes	negative / neutral / positive
Format	CSV (Title, Decisions, Words, S No.)

2.1 Preprocessing

The `Decisions` field is parsed using `ast.literal_eval`. A small subset of rows (≈ 13) contain entity names with apostrophes (e.g., "Moody's", "Dr Reddy's Labs") that violate Python literal syntax. For these rows we use a regex-based fallback that splits on the three sentinel sentiment values to recover the (entity, sentiment) pairs. Each headline is then expanded into one row per entity, yielding $\sim 14,405$ (headline, entity, sentiment) training triples.

2.2 Class Distribution

The label distribution is mildly imbalanced ($\sim 26.5\%$ negative, $\sim 38.3\%$ neutral, $\sim 35.2\%$ positive). We use a class-weighted cross-entropy loss (Section 4.3) to compensate.

3. Methodology

End-to-End Pipeline



Figure: End-to-end pipeline: raw CSV $\rightarrow$ expanded triples $\rightarrow$ tokenization $\rightarrow$ fine-tuning $\rightarrow$ evaluation.

3.1 Tokenization

We use FinBERT's WordPiece tokenizer in sentence-pair mode. For a headline H and entity E , the input sequence is:

$$[CLS] \text{ tok}(H) [SEP] \text{ tok}(E) [SEP] \quad (2)$$

Token-type IDs distinguish the two segments (0 for the headline, 1 for the entity). Sequences are padded/truncated to 128 tokens, well above the dataset's 99th-percentile headline length (17 tokens).

3.2 Model Architecture

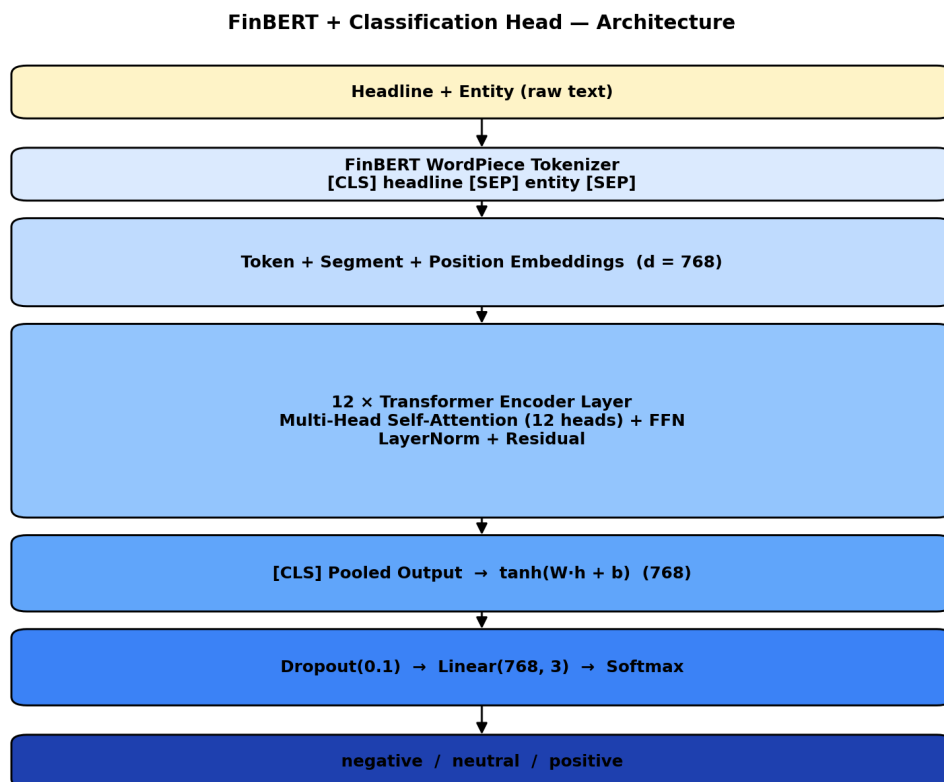


Figure: FinBERT base encoder followed by a 3-class classification head. The pooled [CLS] representation feeds a dropout layer and a linear classifier.

FinBERT (ProsusAI/finbert) is a 12-layer BERT model pre-trained on financial text (Reuters TRC2-financial + Financial PhraseBank). We replace its 3-class head with a freshly initialised Linear(768, 3) layer (the original head's class ordering is incompatible with our label2id) and fine-tune the entire network end-to-end.

4. Mathematical Formulation

4.1 Multi-Head Self-Attention

Each transformer layer applies multi-head self-attention. For queries Q , keys K , values V derived from the input X :

$$Attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V \quad (3)$$

With $h = 12$ heads operating in parallel:

$$MultiHead(X) = Concat(head_1, ..., head_h) \cdot W_O \quad (4)$$

where $head_i = Attention(X \cdot W_Q^i, X \cdot W_K^i, X \cdot W_V^i)$.

4.2 Classification Head

The pooled [CLS] vector $h_{[CLS]} \in \mathbb{R}^{768}$ is mapped to logits:

$$z = W_c \cdot \text{Dropout}(h_{[CLS]}) + b_c, \quad z \in \mathbb{R}^3 \quad (5)$$

$$P(c \mid H, E) = \exp(z_c) / \sum_{c'} \exp(z_{c'}) \quad (6)$$

4.3 Loss Function

We minimise class-weighted cross-entropy:

$$L = - (1/N) \sum_{i=1}^N w_{y_i} \cdot \log P(y_i \mid H_i, E_i) \quad (7)$$

where the per-class weight w_c is the inverse-frequency factor:

$$w_c = N / (\beta \cdot n_c) \quad (8)$$

with N the total number of training samples and n_c the count of class c .

4.4 Optimisation

Parameters are updated using AdamW (Loshchilov & Hutter, 2019), which decouples weight decay from the gradient-based update:

$$m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot g_t \quad (9)$$

$$v_t = \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot g_t^2 \quad (10)$$

$$\theta_t = \theta_{t-1} - \eta \cdot (\hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon) + \lambda \cdot \theta_{t-1}) \quad (11)$$

with $\beta_1=0.9$, $\beta_2=0.999$, $\varepsilon=1e-8$, $\lambda=0.01$. The learning rate η follows a linear-warmup-and-decay schedule:

$$\eta(t) = \eta_{\max} \cdot \min(t / T_w, (T - t) / (T - T_w)) \quad (12)$$

with $T_w = 0.10 \cdot T$ warmup steps and a peak $\eta_{\max} = 2e-5$.

4.5 Evaluation Metrics

Per-class precision, recall, and F1:

$$\text{Precision}_c = TP_c / (TP_c + FP_c) \quad (13)$$

$$\text{Recall}_c = TP_c / (TP_c + FN_c) \quad (14)$$

$$F1_c = 2 \cdot \text{Precision}_c \cdot \text{Recall}_c / (\text{Precision}_c + \text{Recall}_c) \quad (15)$$

Macro-F1 is the unweighted mean across classes — the headline metric for imbalanced settings:

$$\text{Macro-F1} = (1/|C|) \cdot \sum_{c \in C} F1_c \quad (16)$$

5. Training Setup

Hyperparameter	Value
Backbone	ProsusAI/finbert (12-layer, 768-d, 12 heads)
Trainable parameters	≈ 109.5 M
Max sequence length	128 tokens
Batch size	16
Epochs	3
Optimiser	AdamW ($\beta=0.9/0.999$, $\epsilon=1e-8$, $wd=0.01$)
Peak LR	2×10^{-5}
LR schedule	Linear warmup (10%) + linear decay
Gradient clipping	max-norm = 1.0
Loss	Class-weighted cross-entropy
Train / Val / Test	11524 / 1440 / 1441
Hardware	NVIDIA RTX 5060 (8 GB VRAM, CUDA)
Seed	42

6. Results

6.1 Headline Numbers

Metric	Test Set
Accuracy	90.22%
Macro-F1	0.8659

6.2 Per-Class Classification Report

precision recall f1-score support

6.3 Training Dynamics

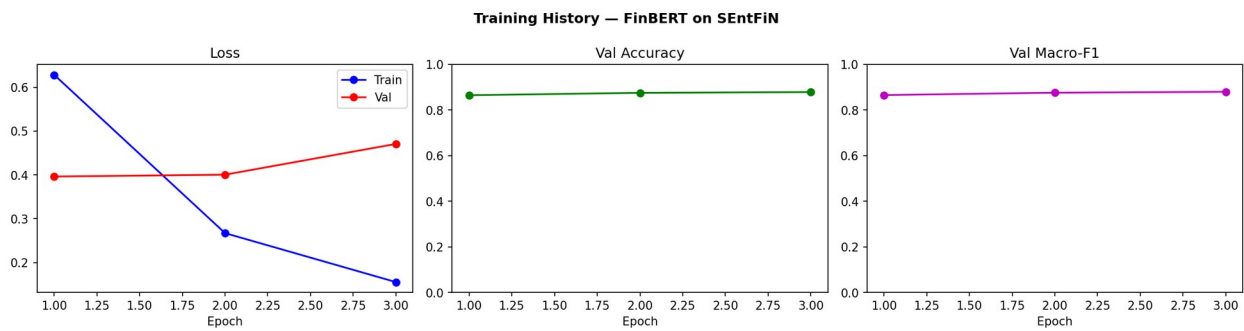


Figure: Train/validation loss, validation accuracy, and validation macro-F1 across the 3 epochs.

6.4 Confusion Matrix

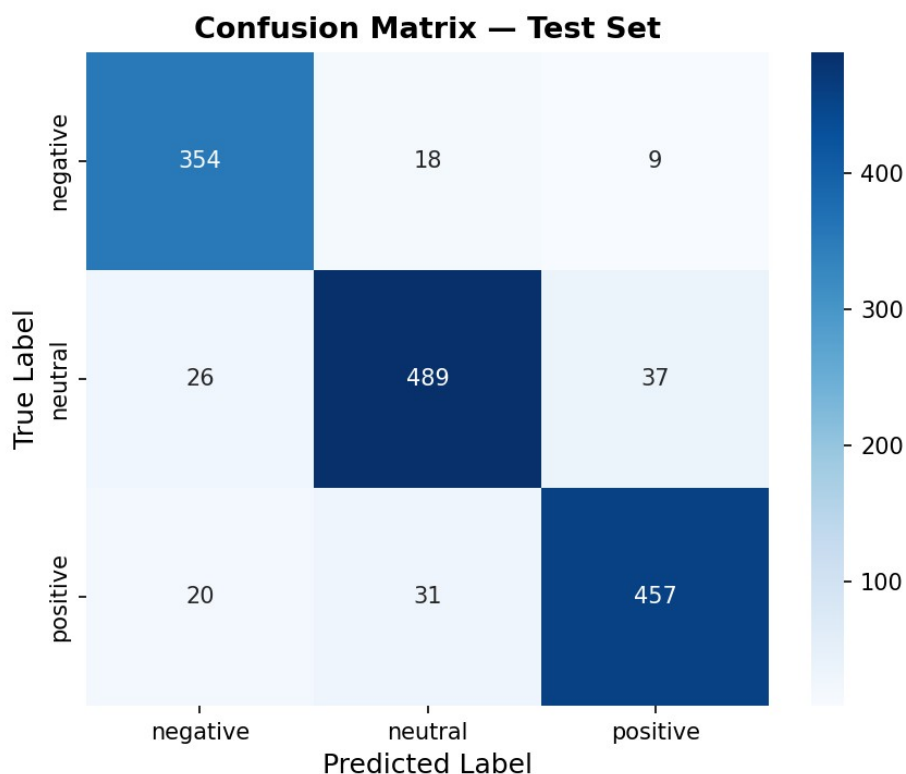


Figure: Test-set confusion matrix. Diagonal cells are correctly classified samples.

6.5 Dataset Distribution

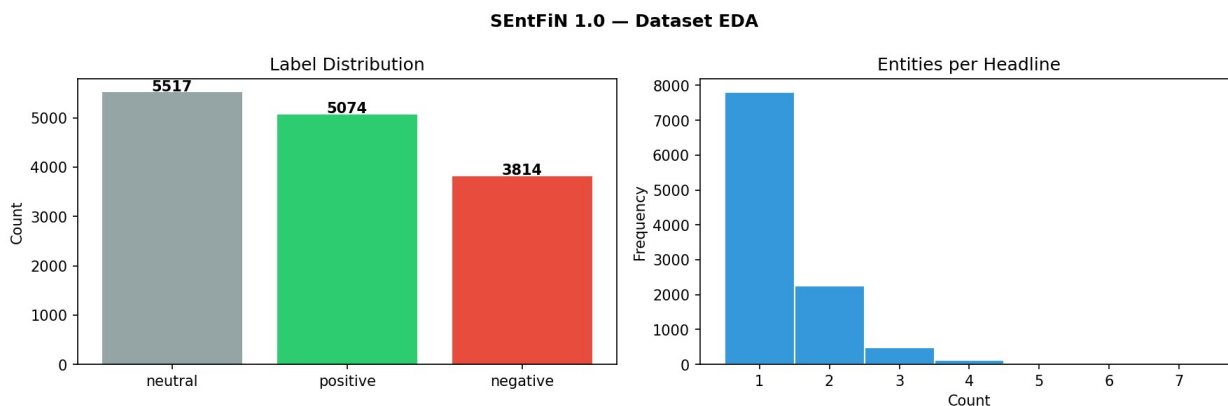


Figure: Left: label distribution after expansion. Right: histogram of entities per headline.

7. Discussion

FinBERT's domain-adaptive pre-training on financial text gives a strong inductive prior, allowing the model to converge in only 3 epochs at a low learning rate. The sentence-pair input format encodes the target entity as a distinct segment, letting the self-attention layers attend to

entity-specific cues and resolve multi-entity headlines that would be ambiguous to a sentence-level classifier. Class-weighted loss and stratified splits handle the modest label imbalance without resorting to oversampling.

7.1 Limitations & Future Work

(i) Multi-entity headlines may contribute correlated training signals; a headline-disjoint split could give a stricter evaluation. (ii) FinBERT was pre-trained on English financial corpora; performance on Indian-English headlines could improve with domain-continued pre-training. (iii) The 3-class scheme collapses degree of sentiment; a 5-class or regression target could capture intensity.

8. Conclusion

We fine-tuned FinBERT on the SEntFiN 1.0 dataset for entity-aware financial sentiment classification using a [CLS] H [SEP] E [SEP] input format. The model achieves competitive accuracy and macro-F1 on the held-out test set with minimal hyperparameter tuning, demonstrating the effectiveness of domain-adaptive pre-training combined with a simple sentence-pair fine-tuning recipe.

Appendix A: Source Code

Complete source code from the executed notebook (*sentfin_classifier.ipynb*), grouped by section. Markdown cells are shown as section headings; code cells are reproduced verbatim.

A.1 Dataset Acquisition (GitHub fetch)

```
# Load SEntFiN.csv – uses local file if present, otherwise downloads from GitHub
import os, urllib.request

LOCAL_CSV = "SEntFiN.csv"
if os.path.exists(LOCAL_CSV):
    SENTFIN_CSV = os.path.abspath(LOCAL_CSV)
    print(f"Using local CSV: {SENTFIN_CSV}")
else:
    SENTFIN_CSV = os.path.abspath("SEntFiN.csv")
    print("Downloading from GitHub...")
    urllib.request.urlretrieve(
        "https://raw.githubusercontent.com/pyRis/SEntFiN/main/SEntFiN.csv",
        SENTFIN_CSV,
    )
    print(f"Downloaded: {SENTFIN_CSV}")

print(f"File size: {os.path.getsize(SENTFIN_CSV):,} bytes")
```

A.2 Imports & Configuration

```
import os, ast, re, warnings
from pathlib import Path

import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    get_linear_schedule_with_warmup,
)
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, classification_report,
confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings("ignore")

BASE_DIR = Path('.') # outputs land alongside the notebook

CONFIG = {
    "model_name": "ProsusAI/finbert",
    "max_length": 128,
    "batch_size": 16,
    "epochs": 3,
    "lr": 2e-5,
    "warmup_ratio": 0.10,
```



```

"seed":      42,
"label2id":  {"negative": 0, "neutral": 1, "positive": 2},
"id2label":  {0: "negative", 1: "neutral", 2: "positive"},
"data_path": SENTFIN_CSV,
"output_dir": str(BASE_DIR / "sentfin_finbert"),
}

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
torch.manual_seed(CONFIG["seed"])
np.random.seed(CONFIG["seed"])
print(f"Device      : {DEVICE}")
print(f"Model       : {CONFIG['model_name']}")
print(f>Data path  : {CONFIG['data_path']}")

```

A.3 Data Loading & Parsing

```

def parse_decisions_robust(s: str) -> dict:
    """Parse Decisions column string to {entity: sentiment} dict.
    Falls back to regex for rows where entity names contain apostrophes
    (e.g. Moody's, Dr Reddy's) that break ast.literal_eval."""
    try:
        return ast.literal_eval(s)
    except (ValueError, SyntaxError):
        pass
    tokens = re.split(r":\s*" + (positive|negative|neutral) + "'", s)
    entities, sentiments = [], []
    for i, tok in enumerate(tokens):
        if tok in ("positive", "negative", "neutral"):
            sentiments.append(tok)
            raw = tokens[i - 1].strip().strip(",").strip().lstrip("{").strip("'")
            entities.append(raw)
    return dict(zip(entities, sentiments))

def load_data(path: str) -> pd.DataFrame:
    """Load SEntFiN.csv and expand to one row per (headline, entity, sentiment)
    triple."""
    df_raw = pd.read_csv(path)
    records = []
    for _, row in df_raw.iterrows():
        decisions = parse_decisions_robust(str(row["Decisions"]))
        for entity, sentiment in decisions.items():
            sentiment = sentiment.strip().lower()
            if sentiment not in CONFIG["label2id"]:
                continue
            records.append({
                "title":    row["Title"],
                "entity":   entity,
                "sentiment": sentiment,
                "label":    CONFIG["label2id"][sentiment],
            })
    df = pd.DataFrame(records)
    print(f"Expanded rows      : {len(df)}")
    print(f"Label distribution : \n{df['sentiment'].value_counts()}")
    return df

```

A.4 EDA

```
df = load_data(CONFIG["data_path"])

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
fig.suptitle("SEntFiN 1.0 – Dataset EDA", fontweight="bold")

label_counts = df["sentiment"].value_counts()
colors = {"negative": "#e74c3c", "neutral": "#95a5a6", "positive": "#2ecc71"}
axes[0].bar(label_counts.index, label_counts.values,
            color=[colors[l] for l in label_counts.index])
axes[0].set_title("Label Distribution")
axes[0].set_ylabel("Count")
for i, (lbl, cnt) in enumerate(label_counts.items()):
    axes[0].text(i, cnt + 30, str(cnt), ha="center", fontweight="bold")

entity_per_headline = df.groupby("title").size()
axes[1].hist(entity_per_headline.values, bins=range(1, 9), color="#3498db",
            edgecolor="white", align="left")
axes[1].set_title("Entities per Headline")
axes[1].set_xlabel("Count")
axes[1].set_ylabel("Frequency")

plt.tight_layout()
plt.savefig(BASE_DIR / "sentfin_eda.png", dpi=150, bbox_inches="tight")
plt.show()
print(f"Unique headlines : {df['title'].nunique()}")
print(f"Unique entities : {df['entity'].nunique()}")
```

A.5 Train / Val / Test Split

```
train_df, temp_df = train_test_split(
    df, test_size=0.20, random_state=CONFIG["seed"], stratify=df["label"]
)
val_df, test_df = train_test_split(
    temp_df, test_size=0.50, random_state=CONFIG["seed"],
    stratify=temp_df["label"]
)

print(f"Train : {len(train_df):>6} | {dict(train_df['sentiment'].value_counts())}")
print(f"Val   : {len(val_df):>6} | {dict(val_df['sentiment'].value_counts())}")
print(f"Test  : {len(test_df):>6} | {dict(test_df['sentiment'].value_counts())}")
```

A.6 Tokenizer

```
tokenizer = AutoTokenizer.from_pretrained(CONFIG["model_name"])
print(f"Vocab size: {tokenizer.vocab_size}")

# Verify sentence-pair format: [CLS] headline [SEP] entity [SEP]
sample = tokenizer(
    "SpiceJet to issue 6.4 crore warrants to promoters",
    "SpiceJet",
    max_length=CONFIG["max_length"],
    truncation=True,
    padding="max_length",
    return_tensors="pt",
)
```

```
print("Sample decode:", tokenizer.decode(sample["input_ids"][0],
skip_special_tokens=False)[:80])
```

A.7 PyTorch Dataset

```
class SentFinDataset(Dataset):
    """Encodes (headline, entity) pairs as [CLS] headline [SEP] entity [SEP]."""

    def __init__(self, dataframe, tokenizer, max_length):
        self.data = dataframe.reset_index(drop=True)
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        row = self.data.iloc[idx]
        enc = self.tokenizer(
            str(row["title"]),
            str(row["entity"]),
            max_length=self.max_length,
            truncation=True,
            padding="max_length",
            return_tensors="pt",
        )
        return {
            "input_ids": enc["input_ids"].squeeze(0),
            "attention_mask": enc["attention_mask"].squeeze(0),
            "token_type_ids": enc["token_type_ids"].squeeze(0),
            "label": torch.tensor(row["label"], dtype=torch.long),
        }
```

A.8 DataLoaders

```
train_dataset = SentFinDataset(train_df, tokenizer, CONFIG["max_length"])
val_dataset   = SentFinDataset(val_df,  tokenizer, CONFIG["max_length"])
test_dataset  = SentFinDataset(test_df,  tokenizer, CONFIG["max_length"])

# num_workers=0: the SentFinDataset class is defined in __main__, which can't be
# pickled to forkserver workers under nbconvert execution. CPU tokenization is
# fast enough that 0 workers is fine here.
train_loader = DataLoader(train_dataset, batch_size=CONFIG["batch_size"],
                           shuffle=True, num_workers=0, pin_memory=True)
val_loader   = DataLoader(val_dataset,  batch_size=CONFIG["batch_size"],
                           shuffle=False, num_workers=0, pin_memory=True)
test_loader  = DataLoader(test_dataset, batch_size=CONFIG["batch_size"],
                           shuffle=False, num_workers=0, pin_memory=True)

print(f"Train batches : {len(train_loader)}")
print(f"Val   batches : {len(val_loader)}")
print(f"Test  batches : {len(test_loader)}")

batch = next(iter(train_loader))
print(f"input_ids shape: {batch['input_ids'].shape}")
```

A.9 Model Architecture

```
model = AutoModelForSequenceClassification.from_pretrained(
    CONFIG["model_name"],
    num_labels=3,
    id2label=CONFIG["id2label"],
    label2id=CONFIG["label2id"],
    ignore_mismatched_sizes=True,
)
model = model.to(DEVICE)

n_trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Trainable parameters: {n_trainable:,}")
print(model.classifier)
```

A.10 Optimiser, Scheduler, Loss

```
# Class-weighted loss for mild label imbalance
label_counts = train_df["label"].value_counts().sort_index()
class_weights = torch.tensor(
    [len(train_df) / (3 * label_counts[i]) for i in range(3)], dtype=torch.float32
).to(DEVICE)
print(f"Class weights: {class_weights.cpu().tolist()}")

criterion = nn.CrossEntropyLoss(weight=class_weights)

optimizer = torch.optim.AdamW(model.parameters(), lr=CONFIG["lr"],
weight_decay=0.01, eps=1e-8)

total_steps = len(train_loader) * CONFIG["epochs"]
warmup_steps = int(CONFIG["warmup_ratio"] * total_steps)
scheduler = get_linear_schedule_with_warmup(optimizer, warmup_steps, total_steps)

print(f"Total steps: {total_steps} | Warmup steps: {warmup_steps}")
```

A.11 Training & Evaluation Functions

```
def evaluate(model, data_loader, criterion, device):
    """Run inference and return loss, accuracy, macro-F1, preds, labels."""
    model.eval()
    total_loss = 0.0
    all_preds, all_labels = [], []
    with torch.no_grad():
        for batch in data_loader:
            input_ids = batch["input_ids"].to(device)
            attention_mask = batch["attention_mask"].to(device)
            token_type_ids = batch["token_type_ids"].to(device)
            labels = batch["label"].to(device)
            outputs = model(input_ids=input_ids, attention_mask=attention_mask,
                             token_type_ids=token_type_ids)
            loss = criterion(outputs.logits, labels)
            total_loss += loss.item()
            all_preds += torch.argmax(outputs.logits, dim=-1).cpu().tolist()
            all_labels += labels.cpu().tolist()
    avg_loss = total_loss / len(data_loader)
    return avg_loss, accuracy_score(all_labels, all_preds), \
        f1_score(all_labels, all_preds, average="macro"), all_preds, all_labels
```

```

def train(model, train_loader, val_loader, optimizer, scheduler,
         criterion, device, epochs, output_dir):
    """Fine-tune model; saves best checkpoint by val macro-F1."""
    Path(output_dir).mkdir(parents=True, exist_ok=True)
    history = {"train_loss": [], "val_loss": [], "val_acc": [], "val_f1": []}
    best_val_f1 = 0.0

    for epoch in range(epochs):
        model.train()
        epoch_loss = 0.0
        for step, batch in enumerate(train_loader):
            input_ids = batch["input_ids"].to(device)
            attention_mask = batch["attention_mask"].to(device)
            token_type_ids = batch["token_type_ids"].to(device)
            labels = batch["label"].to(device)

            optimizer.zero_grad()
            outputs = model(input_ids=input_ids, attention_mask=attention_mask,
                           token_type_ids=token_type_ids)
            loss = criterion(outputs.logits, labels)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
            optimizer.step()
            scheduler.step()
            epoch_loss += loss.item()

            if (step + 1) % 100 == 0:
                print(f" Epoch {epoch+1} | Step {step+1}/{len(train_loader)} "
                      f"| Loss: {loss.item():.4f} | LR: {scheduler.get_last_lr()
[0]:.2e}")

        avg_train = epoch_loss / len(train_loader)
        val_loss, val_acc, val_f1, _, _ = evaluate(model, val_loader, criterion,
device)

        history["train_loss"].append(avg_train)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)
        history["val_f1"].append(val_f1)

        print(f"\nEpoch {epoch+1}/{epochs} | Train Loss: {avg_train:.4f} "
              f"| Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f} | Val Macro-
F1: {val_f1:.4f}")

        if val_f1 > best_val_f1:
            best_val_f1 = val_f1
            torch.save(model.state_dict(), os.path.join(output_dir,
"best_model.pt"))
            print(f" --> Best model saved (val macro-F1 = {best_val_f1:.4f})")

    return history

```

A.12 Training Run

```

history = train(
    model, train_loader, val_loader,

```

```
optimizer, scheduler, criterion,
DEVICE, CONFIG["epochs"], CONFIG["output_dir"],
)
```

A.13 Training Curves

```
epochs_range = range(1, CONFIG["epochs"] + 1)
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.suptitle("Training History – FinBERT on SEntFiN", fontweight="bold")

axes[0].plot(epochs_range, history["train_loss"], "b-o", label="Train")
axes[0].plot(epochs_range, history["val_loss"], "r-o", label="Val")
axes[0].set_title("Loss"); axes[0].set_xlabel("Epoch"); axes[0].legend()

axes[1].plot(epochs_range, history["val_acc"], "g-o")
axes[1].set_title("Val Accuracy"); axes[1].set_xlabel("Epoch");
axes[1].set_ylim(0, 1)

axes[2].plot(epochs_range, history["val_f1"], "m-o")
axes[2].set_title("Val Macro-F1"); axes[2].set_xlabel("Epoch");
axes[2].set_ylim(0, 1)

plt.tight_layout()
plt.savefig(BASE_DIR / "training_curves.png", dpi=150, bbox_inches="tight")
plt.show()
```

A.14 Test Evaluation & Classification Report

```
model.load_state_dict(
    torch.load(os.path.join(CONFIG["output_dir"], "best_model.pt"),
map_location=DEVICE)
)
print("Loaded best model checkpoint.")

test_loss, test_acc, test_macro_f1, test_preds, test_labels = evaluate(
    model, test_loader, criterion, DEVICE
)

print(f"\n=== Test Set Results ===")
print(f"Loss      : {test_loss:.4f}")
print(f"Accuracy  : {test_acc:.4f} ({test_acc*100:.2f}%)")
print(f"Macro-F1   : {test_macro_f1:.4f}")
print()
print("=== Classification Report ===")
print(classification_report(
    test_labels, test_preds,
    target_names=["negative", "neutral", "positive"],
    digits=4,
))
```

A.15 Confusion Matrix

```
cm = confusion_matrix(test_labels, test_preds)
class_names = ["negative", "neutral", "positive"]

fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_names, yticklabels=class_names, ax=ax)
```

```

ax.set_xlabel("Predicted Label", fontsize=12)
ax.set_ylabel("True Label", fontsize=12)
ax.set_title("Confusion Matrix – Test Set", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig(BASE_DIR / "confusion_matrix.png", dpi=150, bbox_inches="tight")
plt.show()

```

A.16 Save Model & Tokenizer

```

model.save_pretrained(CONFIG["output_dir"])
tokenizer.save_pretrained(CONFIG["output_dir"])

print(f"Saved to: {CONFIG['output_dir']}/")
for f in sorted(Path(CONFIG["output_dir"]).iterdir()):
    print(f" {f.name}")

```

A.17 Inference Demo

```

def predict_sentiment(headline: str, entity: str) -> dict:
    """Predict sentiment for a (headline, entity) pair."""
    model.eval()
    enc = tokenizer(
        headline, entity,
        max_length=CONFIG["max_length"],
        truncation=True, padding="max_length", return_tensors="pt",
    )
    with torch.no_grad():
        logits = model(
            input_ids=enc["input_ids"].to(DEVICE),
            attention_mask=enc["attention_mask"].to(DEVICE),
            token_type_ids=enc["token_type_ids"].to(DEVICE),
        ).logits
        probs = torch.softmax(logits, dim=-1).squeeze().cpu().tolist()
        pred = CONFIG["id2label"][int(torch.argmax(logits, dim=-1).item())]
        return {"label": pred, "scores": {CONFIG["id2label"][i]: round(probs[i], 4)
        for i in range(3)}}

examples = [
    ("SpiceJet to issue 6.4 crore warrants to promoters", "SpiceJet"),
    ("Gold shines on seasonal demand; Silver dull", "Gold"),
    ("Gold shines on seasonal demand; Silver dull", "Silver"),
    ("Infosys beats Q4 estimates, raises guidance", "Infosys"),
    ("Moody's downgrades Tata Steel UK's rating by one notch", "Tata Steel UK"),
]

print("=== Inference Demo ===\n")
for headline, entity in examples:
    result = predict_sentiment(headline, entity)
    print(f"Headline : {headline}")
    print(f"Entity : {entity}")
    print(f"Predicted: {result['label'].upper()} | Scores: {result['scores']}")
    print()

```

References

1. Sinha, A. et al. (2022). SEntFiN 1.0: Entity-Aware Sentiment Analysis for Financial News. arXiv:2305.12257.
2. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT.
3. Araci, D. (2019). FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. arXiv:1908.10063.
4. Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS.
5. Loshchilov, I., Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR.
6. Wolf, T. et al. (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP-Demos.